

# **“Lost and Found Items Tracker for Campus”**

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

## **BACHELOR OF TECHNOLOGY**

**In**

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**

**By**

**KAVI KRISHNA**

**2520080021**

**HEMANTH THUMU**

**2520090074**

**PRAJWAL**

**2520080017**

Under the Esteemed Guidance of

**Khaja Shareef**

**Department of Computer Science and Information Technology**



**Koneru Lakshmaiah Education Foundation**

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, [www.klh.edu.in](http://www.klh.edu.in)

**K L (Deemed to be) University**

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**



## **Declaration**

The Project Report entitled **“Lost and Found Items Tracker for Campus”** is a record of bonafide work of **KAVI KRISHNA – 2520080021, HEMANTH THUMU – 2520090074, PRAJWAL – 2520080017** submitted in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 12, Team ID: \_\_\_\_\_, during 2026–2027, Trimester 4, at KLH University. The results embodied in this report have not been copied from any other student, department, or institution.

**KAVI KRISHNA – 2520080021**

**HEMANTH THUMU – 2520090074**

**PRAJWAL – 2520080017**

**K L (Deemed to be) University**

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**



## **CERTIFICATE**

This is to certify that the project report entitled **“Lost and Found Items Tracker for Campus”** is a bonafide work done and submitted by **KAVI KRISHNA – 2520080021, HEMANTH THUMU – 2520090074, PRAJWAL – 2520080017** in partial fulfillment of the requirements of the course **DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E)**, Section 12, Team ID: \_\_\_\_\_, at KLH University during 2026–2027, Trimester 4.

**Signature of the Guide**

**Signature of the Course Coordinator**

**Signature of the HOD**

## **ACKNOWLEDGEMENT**

The success of this project would not have been possible without the timely help and guidance of many people. We would like to express our sincere thanks to everyone who assisted us, in one way or another, in the completion of our project.

Our greatest appreciation goes to our Course Coordinator and to our guide, Khaja Shareef, Department of Computer Science and Information Technology, whose tremendous support, encouragement and guidance throughout this project cannot be expressed in words.

We express our gratitude to the Head of the Department of Computer Science and Information Technology for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

**KAVI KRISHNA – 2520080021**

**HEMANTH THUMU – 2520090074**

**PRAJWAL – 2520080017**

## **Abstract**

The Lost and Found Items Tracker for Campus is a centralized web platform that allows students, faculty and staff to report, search, track and recover lost and found belongings within a college campus. Today, lost-and-found handling on most campuses is manual and decentralized — notice boards, scattered messaging groups and word of mouth — and it offers no location context and no reliable way of verifying that an item is handed back to its rightful owner.

The proposed system provides a single web interface through which any campus user can report a lost or found item, search and filter existing reports, and see where an item was lost or found on an interactive, floor-by-floor campus map. A match-scoring engine compares reports using their category, description, date and location and automatically suggests likely matches. Owners submit claims for items, and administrators verify each claim before the item is handed over, so that recovery is secure and accountable.

The application uses an HTML5, CSS3 and JavaScript frontend, Node.js REST APIs for the business logic, and a MongoDB database (hosted on MongoDB Atlas) for persistent storage of user accounts, lost and found items, claims and item status. Role-based authentication separates ordinary campus users from administrators. Every item moves through a simple lifecycle — Lost, Found, Returned — so its state is always visible. The system achieves its aim of making recovery faster, searchable and location-aware, replacing notice boards and scattered groups with one verified, centralized platform.

## Index

| S.No. | Chapters                            | Topics                                                                                                                                                                   | Page No. |
|-------|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
|       | <b>Acknowledgement</b>              |                                                                                                                                                                          | 4        |
|       | <b>Abstract</b>                     |                                                                                                                                                                          | 5        |
| 1     | <b>Introduction</b>                 | 1.1 Background of the Project<br>1.2 Problem Statement<br>1.3 Scope of the Project<br>1.4 Objectives<br>1.5 Importance of the Application<br>1.6 Target Users / Audience | 8        |
| 2     | <b>System Requirements</b>          | 2.1 Hardware Requirements<br>2.2 Software Requirements<br>2.3 Development Tools and Frameworks                                                                           | 10       |
| 3     | <b>Technology Stack</b>             | 3.1 Frontend<br>3.2 Backend<br>3.3 Database<br>3.4 Version Control<br>3.5 Authentication and Security                                                                    | 11       |
| 4     | <b>System Architecture</b>          | 4.1 High-Level Architecture Diagram<br>4.2 Description of Each Layer<br>4.3 Deployment Architecture                                                                      | 12       |
| 5     | <b>Design</b>                       | 5.1 Database Design Principles<br>5.2 ER Diagram / Database Schema<br>5.3 Design Decisions and Assumptions                                                               | 13       |
| 6     | <b>Implementation</b>               | 6.1 Module-wise Implementation<br>6.2 Front-end Logic<br>6.3 API Integration<br>6.4 Authentication and Authorization                                                     | 14       |
| 7     | <b>Features</b>                     | 7.1 List of Main Features<br>7.2 How Each Feature Works                                                                                                                  | 16       |
| 8     | <b>Testing</b>                      | 8.1 API Testing<br>8.2 Integration Testing<br>8.3 Tools Used for Testing<br>8.4 Test Cases and Results                                                                   | 17       |
| 9     | <b>Deployment</b>                   | 9.1 Steps to Deploy<br>9.2 Environment Configuration<br>9.3 Hosting of Frontend and Backend                                                                              | 18       |
| 10    | <b>Challenges &amp; Limitations</b> | 10.1 Issues Faced During Development<br>10.2 Solutions Applied<br>10.3 Current Limitations or Known Bugs                                                                 | 19       |

| S.No. | Chapters                   | Topics                                                                         | Page No. |
|-------|----------------------------|--------------------------------------------------------------------------------|----------|
| 11    | <b>Future Enhancements</b> | 11.1 Planned Features<br>11.2 Possible Integrations or Optimizations           | 20       |
| 12    | <b>Conclusion</b>          | 12.1 Summary of the Project<br>12.2 What Was Achieved<br>12.3 Skills Learned   | 21       |
| 13    | <b>References</b>          | Documentation, tools, and resources used                                       | 22       |
| 14    | <b>Appendices</b>          | Sample Logic<br>Installation / Setup Instructions<br>User Roles<br>Screenshots | 23       |

# CHAPTER 1: INTRODUCTION

## 1.1 Background of the Project

A college campus is a large, busy environment in which students, faculty and staff carry laptops, ID cards, water bottles, books, keys, phones and many other personal items from one class, lab and canteen to another. Items are inevitably misplaced, and when this happens the owner usually has no clear place to turn to. In most institutions, the process of reporting and recovering lost belongings is informal: a notice pinned on a board, a message forwarded to several group chats, or a visit to whichever office might have found the item.

This approach works only when activity is low. Messages are buried within hours, notices are seen by only a few people, nobody records where an item was actually found, and there is no reliable way to check that the person collecting an item is really its owner. The Lost and Found Items Tracker for Campus was built to replace this scattered handling with one centralized, searchable and verified platform.

## 1.2 Problem Statement

Campus lost-and-found handling is manual, decentralized and lacks location context or verification. Given a campus with many buildings, floors and rooms, and a continuous stream of lost and found reports from different people, the system must answer three questions reliably: where was an item lost or found, which found item is most likely to be the one a person has lost, and whether the person claiming an item is really its owner. Answering these questions manually does not scale, is error-prone, and leaves owners with no visibility of what has been found.

## 1.3 Scope of the Project

The system is intended for campus students, faculty and staff and covers the full recovery cycle of a lost or found item: reporting an item together with its location; searching and filtering reports; tracking items on an interactive campus map; receiving automated match suggestions based on category, description, date and location; submitting a claim for an item; and administrator verification of each claim before handover. Features such as real-time push or email alerts, a native mobile application, QR-code tagging, ID-card integration and image-based matching were treated as out of scope for this submission and are listed under Future Enhancements.

## 1.4 Objectives

- **Objective 1:** Provide one centralized platform for reporting lost and found items.
- **Objective 2:** Enable location-based tracking with an interactive campus map.
- **Objective 3:** Automatically suggest matches using category, description, date and location.
- **Objective 4:** Support secure handover of items through claims and administrator verification.
- **Objective 5:** Restrict sensitive actions to administrators through role-based access.

## **1.5 Importance of the Application**

By centralizing all reports in one place, the system replaces notice boards and scattered groups with a single searchable record. Location-tagged reports tell an owner exactly where to look, match suggestions shorten the time needed to find the right item, and administrator verification protects owners by ensuring that a found item is released only to the person who can prove it is theirs. For the institution, the status of every item (Lost, Found or Returned) is always visible, which makes the whole process accountable.

## **1.6 Target Users / Audience**

The system serves campus users and administrators. Students, faculty and staff use it to report lost or found items, search and filter reports, view locations on the campus map, review match suggestions and submit claims. Administrators use it to review reports, verify claims, hand over items and keep the records accurate.

## **CHAPTER 2: SYSTEM REQUIREMENTS**

### **2.1 Hardware Requirements**

The application has modest hardware requirements. A development or demonstration machine with a dual-core processor, 4 GB of RAM and a few hundred megabytes of free disk space (for Node.js modules and project source) is sufficient to run the frontend and backend. The database is hosted on MongoDB Atlas, so no local database server is required; an internet connection is needed to reach it.

### **2.2 Software Requirements**

- Node.js (current LTS release) with npm
- MongoDB database (MongoDB Atlas cloud cluster)
- A modern web browser: Chrome, Firefox or Edge
- Git, for version control
- Visual Studio Code, for development

### **2.3 Development Tools and Frameworks**

The frontend was written in plain HTML5, CSS3 and JavaScript without a frontend framework. The backend is a set of Node.js REST APIs, and MongoDB is used as the database. Visual Studio Code was the primary development environment and Git was used for source control.

## CHAPTER 3: TECHNOLOGY STACK

### 3.1 Frontend

The frontend is built with HTML5, CSS3 and JavaScript. It provides the login page for students and administrators, forms for reporting lost and found items, search and filter controls, the interactive campus map, the match-suggestion view, the claim forms and the administrator dashboard. The interface uses a clean monochrome black-and-white theme with soft, colorful ambient background effects, and works in current versions of Chrome, Firefox and Edge.

### 3.2 Backend

The backend consists of Node.js REST APIs. They receive requests from the frontend, authenticate the user, apply the business rules (for example, matching and claim verification) and read from or write to the database. The core backend functions are role-based authentication, a match-scoring engine and claim verification.

### 3.3 Database

MongoDB stores all persistent data: user accounts, lost and found items, claims and item status. Early prototypes of the application kept data in the browser's localStorage; moving the data layer to MongoDB Atlas allows records to persist and synchronize automatically for every user instead of staying on a single device.

### 3.4 Version Control

Git was used throughout development for version control, so that the three team members could work on the project in parallel and keep a history of changes.

### 3.5 Authentication and Security

The system uses role-based authentication. Users sign in with their credentials, and the role attached to the account (campus user or administrator) decides what the user may do. Administrator-only functions, such as claim verification and handover, are not available to ordinary campus users.

## CHAPTER 4: SYSTEM ARCHITECTURE

### 4.1 High-Level Architecture Diagram

The system follows a standard three-tier architecture that separates the user interface, the business logic and the stored data. Figure 4.1 shows the three layers and how a request flows between them.

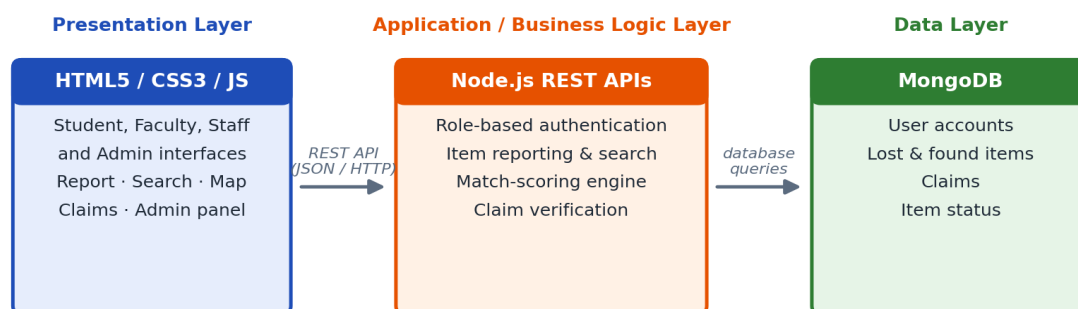


Figure 4.1: High-level three-tier architecture of the Lost and Found Items Tracker

### 4.2 Description of Each Layer

The Presentation Layer is the web interface written in HTML5, CSS3 and JavaScript. It never accesses the database directly; every piece of data it shows is requested from the REST APIs. Users of all kinds — students, faculty, staff and administrators — interact with the system through this layer.

The Application Layer is the set of Node.js REST APIs. Incoming requests are authenticated and checked against the user’s role, then passed to the relevant logic: reporting and searching items, computing match scores, and handling and verifying claims.

The Data Layer is the MongoDB database, which stores user accounts, lost and found items, claims and item status.

### 4.3 Deployment Architecture

For development and demonstration, the frontend and the backend run on the developer’s machine while the database is hosted on MongoDB Atlas. The project was designed for a standard web-server or hosting environment, so the same code can later be moved to a hosted deployment; Chapter 11 outlines further improvements.

## CHAPTER 5: DESIGN

### 5.1 Database Design Principles

Each fact is stored in exactly one place, and relationships between entities are expressed by references (an item stores who reported it; a claim stores which item it is for and who made it) rather than by duplicating data. Validation is applied both in the interface and in the backend so that incomplete or inconsistent records, such as an item without a category or a claim for a non-existent item, are rejected before they are stored.

### 5.2 ER Diagram / Database Schema

The data model has four entities, shown in Figure 5.1: user accounts, lost and found items, claims, and item status.

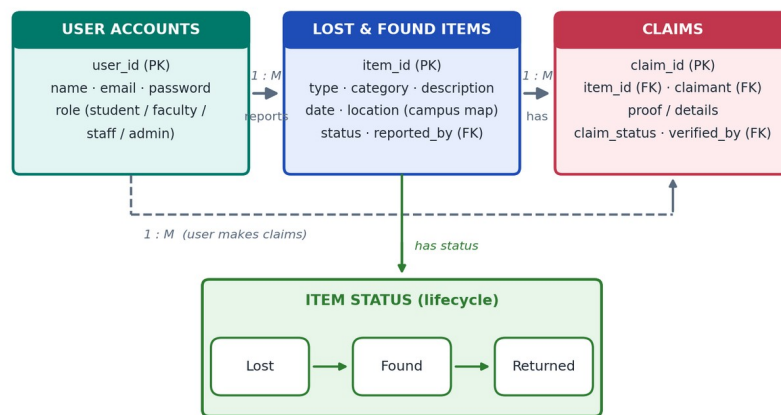


Figure 5.1: Entity-relationship diagram of the Lost and Found Items Tracker

- **USER ACCOUNTS** store every account with its credentials and role (student, faculty, staff or administrator).
- **LOST & FOUND ITEMS** hold each report: whether it is a lost or found item, its category, description, date, location on the campus map, current status and the user who reported it.
- **CLAIMS** link a user to an item they claim to own, together with the details supplied as proof and the claim's verification outcome.
- **ITEM STATUS** records where an item is in its lifecycle: Lost → Found → Returned.

### 5.3 Design Decisions and Assumptions

- **Status as a lifecycle:** an item's state is a single value that only moves forward (Lost → Found → Returned), so the current state of any item is unambiguous.
- **Verification before handover:** a claim never changes an item to Returned by itself; only an administrator's verification does.
- **Location from a fixed map:** locations are chosen from the campus map rather than typed freely, which keeps location data consistent and makes location-based matching reliable.
- **Field names:** the attribute names shown in Figure 5.1 describe the logical design; exact keys and cardinalities follow the implemented collections.

## CHAPTER 6: IMPLEMENTATION

### 6.1 Module-wise Implementation

The system is organized into the following modules, shown in order of the workflow in Figure 6.1.

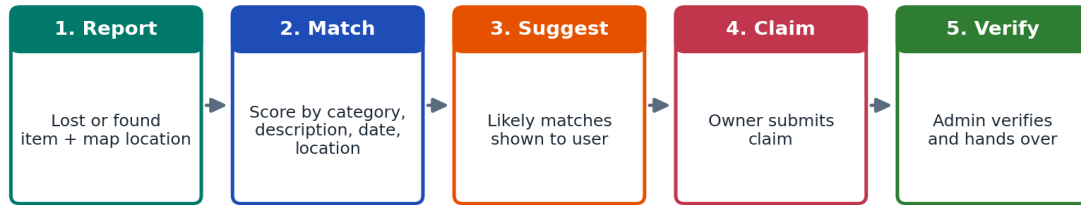


Figure 6.1: Recovery workflow from report to verified handover

- **Authentication:** registration and login for campus users and administrators, with the role stored on the account.
- **Item reporting:** a form to report a lost or found item with its category, description, date and location.
- **Search and filter:** campus users can search reports and filter them by type, category, date and location.
- **Campus map:** an interactive map of the college building, shown floor by floor, on which reports are placed (see Table 6.1).
- **Match-scoring engine:** compares a new report against existing reports using category, description, date and location, and suggests the closest candidates.
- **Claims and verification:** an owner submits a claim for an item; an administrator verifies it and records the handover.
- **Administrator dashboard:** gives administrators one place to review reports and claims and update item status.

Table 6.1: Campus map layout used for location tagging

| Level         | Areas covered             |
|---------------|---------------------------|
| Basement (-1) | Canteen, Labs 14 and 15   |
| Floor 1       | Rooms 101–108             |
| Floor 2       | Library, Rooms 201–206    |
| Floor 3       | Auditorium, Rooms 301–307 |
| Floor 4       | Rooms 401–407             |

The match-scoring engine is deliberately simple to explain: each candidate pair of a lost report and a found report receives a score that grows when the category is the same, when the descriptions share key words, when the dates are close together, and when the locations are near each other on the map. Candidates with the highest scores are suggested first, and the final decision always stays with the owner and the administrator.

## **6.2 Front-end Logic**

The frontend is written in plain JavaScript. Shared interface elements such as the navigation bar, item cards, status badges and dialogs are written once and reused across the pages so that every page loads, reports errors and confirms actions in a consistent way. The visual design uses a black-and-white theme with colorful ambient background effects.

## **6.3 API Integration**

The frontend communicates with the backend only through the REST APIs, sending and receiving JSON. Each kind of data (accounts, items, claims) is handled by its own group of endpoints, so no page needs to know how data is stored. Moving from browser localStorage to MongoDB Atlas therefore changed the data layer without changing how the pages request information.

## **6.4 Authentication and Authorization**

Registration creates an ordinary campus-user account. Administrator privileges are not available through public registration; they are assigned separately so that no one can gain administrator access by signing up. Each protected request is checked for a valid signed-in user, and administrator-only operations — such as verifying a claim or marking an item as Returned — additionally check that the user's role is administrator.

## CHAPTER 7: FEATURES

### 7.1 List of Main Features

#### Campus-user features (students, faculty and staff):

- Report a lost or found item with category, description, date and location
- Search and filter all reports
- View item locations on the interactive floor-by-floor campus map
- Receive automated match suggestions for a report
- Submit a claim for an item and follow its status (Lost → Found → Returned)

#### Administrator features:

- Dashboard of all reports, claims and item statuses
- Verify or reject claims before handover
- Mark items as Found or Returned and keep records accurate
- Manage accounts and monitor activity across the campus

### 7.2 How Each Feature Works

**Reporting:** a user chooses whether the item is lost or found, selects a category, writes a description, enters the date and picks the place on the campus map. The report is stored in the database with the status Lost or Found.

**Search and filter:** the user types a keyword or selects filters; the backend returns only the reports that satisfy them, so the list stays short and relevant.

**Match suggestions:** when a report is created, the match-scoring engine compares it with opposite-type reports (lost against found) using category, description, date and location, and shows the best candidates.

**Claims and verification:** an owner who recognizes an item submits a claim with supporting details. The administrator reviews the claim, verifies it, and only then marks the item as Returned. This protects the real owner from false claims.

## CHAPTER 8: TESTING

### 8.1 API Testing

Each REST endpoint is exercised with a REST client such as Postman, sending valid and invalid requests and checking the HTTP status code and response body. Requests are repeated with a campus-user account and with an administrator account to confirm that role restrictions are enforced.

### 8.2 Integration Testing

Integration testing checks that the frontend, the backend and the MongoDB Atlas database work together as a whole: a report created in the browser must be stored, appear in search, be matched, be claimed and finally be verified and returned, with the item status updating at every step.

### 8.3 Tools Used for Testing

- Postman (or any REST client) — manual API testing
- Browser developer tools — inspecting requests, responses and interface behavior
- MongoDB Atlas data explorer — verifying stored records directly

### 8.4 Test Cases and Results

The table below lists the test cases that cover the main behavior of the system. The Actual Result column is to be filled in with Pass or Fail when the test is executed, since numeric performance and CRUD results were not available at the Review-2 stage.

| #  | Test Case                                                                                    | Expected Result                                  | Actual Result |
|----|----------------------------------------------------------------------------------------------|--------------------------------------------------|---------------|
| 1  | Login with a wrong password                                                                  | Login rejected                                   |               |
| 2  | Campus-user account calls an administrator-only function                                     | Access denied                                    |               |
| 3  | Report an item with a missing required field                                                 | Report rejected with a clear message             |               |
| 4  | Report a lost item and a found item with the same category, close dates and nearby locations | Each appears among the other's match suggestions |               |
| 5  | Search and filter by category and date                                                       | Only matching reports are listed                 |               |
| 6  | Select a floor on the campus map                                                             | Reports for that floor are shown                 |               |
| 7  | Submit a claim for a found item                                                              | Claim stored and shown as pending                |               |
| 8  | Administrator verifies a claim                                                               | Item status changes to Returned                  |               |
| 9  | Campus user tries to mark an item as Returned                                                | Request denied                                   |               |
| 10 | Reload the page after reporting an item                                                      | The record persists (stored in MongoDB)          |               |

## CHAPTER 9: DEPLOYMENT

### 9.1 Steps to Deploy

The application is run locally for development and demonstration as follows:

- Create a MongoDB Atlas cluster and obtain its connection string.
- Place the connection string and other settings in the server's environment file.
- Install the backend dependencies and start the server (npm install, then npm start).
- Open the frontend in a browser and sign in with a registered account.

### 9.2 Environment Configuration

All server configuration — the MongoDB Atlas connection string, the port the server listens on and any secret values — is kept in an environment file that is never committed to version control. A template file with placeholder values can be committed instead, so the required settings are documented without exposing real credentials.

### 9.3 Hosting of Frontend and Backend

For this submission the frontend and the backend run as local processes while the database runs in the MongoDB Atlas cloud. The technology stack is standard, so both parts can be placed on any ordinary web-hosting environment for campus-wide use; this is listed as a future step.

## **CHAPTER 10: CHALLENGES & LIMITATIONS**

### **10.1 Issues Faced During Development**

- Persistence: the first versions stored data in the browser's localStorage, so data stayed on one device and was not shared between users.
- Locations: describing where an item was lost in free text was inconsistent, which made location-based matching unreliable.
- Matching quality: deciding how much weight category, description, date and location should carry in the match score.
- Interface design: the visual theme was reworked from a glass-style design to a cleaner monochrome theme for readability.

### **10.2 Solutions Applied**

The data layer was moved from localStorage to MongoDB Atlas so that records persist and synchronize automatically. A floor-by-floor map of the college building was introduced so that locations are chosen from a fixed set of places instead of free text. The match-scoring engine combines all four attributes so that no single attribute can dominate, and every suggestion is reviewed by a person before any handover. The interface was simplified to the black-and-white theme described in Chapter 3.

### **10.3 Current Limitations or Known Bugs**

- There are no real-time push or email notifications; users must check the system for updates.
- There is no native Android or iOS application.
- Matching relies on text and structured fields; photographs of items are not compared.
- Numeric performance and CRUD test results were not available at the Review-2 stage.
- The system has been run in a local development setup and has not yet been deployed for campus-wide use.

## **CHAPTER 11: FUTURE ENHANCEMENTS**

### **11.1 Planned Features**

- Real-time push and email alerts when a possible match or claim update appears.
- A native Android and iOS application.
- QR-code tagging of belongings so that a found item can be linked to its owner instantly.
- ID-card integration so that lost ID cards can be returned directly to the right person.
- AI image matching to compare photographs of lost and found items.

### **11.2 Possible Integrations or Optimizations**

- Hosting the frontend, backend and database in a public cloud environment for campus-wide access.
- Containerizing the application for consistent, reproducible deployment.
- Adding an automated test suite so that regressions are caught automatically.
- Caching frequently requested searches to reduce database load.

## **CHAPTER 12: CONCLUSION**

### **12.1 Summary of the Project**

This project set out to replace the manual, decentralized handling of lost and found items on a college campus with a single web platform. The tracker combines reporting, interactive campus mapping, automated matching and administrator oversight, and it addresses the manual and decentralized process described in the problem statement.

### **12.2 What Was Achieved**

- A centralized, searchable platform that replaces notice boards and scattered groups.
- Location-tagged reports that improve tracking through the campus map.
- Match suggestions based on category, description, date and location.
- Administrator verification that supports secure handover.
- Role-based access and a persistent MongoDB database behind the interface.

### **12.3 Skills Learned**

Working on this project gave the team practical experience in web development with HTML5, CSS3 and JavaScript; building REST APIs with Node.js; designing and using a MongoDB database and moving an application from browser storage to a cloud database; implementing role-based authentication; designing a simple match-scoring approach; using Git for team collaboration; and testing a full-stack application as a team.

## CHAPTER 13: REFERENCES

- Node.js — Official Documentation, <https://nodejs.org/en/docs>
- MongoDB Manual, <https://www.mongodb.com/docs/manual/>
- MongoDB Atlas Documentation, <https://www.mongodb.com/docs/atlas/>
- MDN Web Docs — HTML, CSS and JavaScript reference, <https://developer.mozilla.org>
- Git — Official Documentation, <https://git-scm.com/doc>
- Visual Studio Code Documentation, <https://code.visualstudio.com/docs>
- Postman Learning Center, <https://learning.postman.com>
- Review-2 project presentation, Lost & Found Items Tracker for Campus (team source).

## CHAPTER 14: APPENDICES

### Appendix A: Match-Scoring Logic

The match-scoring engine follows this outline for every pair of a lost report and a found report:

- Category: the score increases if both reports belong to the same category.
- Description: the score increases with the number of key words the two descriptions share.
- Date: the score increases the closer the two dates are.
- Location: the score increases the nearer the two locations are on the campus map.
- Result: candidates are ranked by total score and the highest-ranked ones are suggested.

### Appendix B: Installation and Setup Instructions

- Create a MongoDB Atlas cluster and copy its connection string.
- Add the connection string to the server's environment file.
- In the server folder, run npm install and then npm start.
- Open the frontend in Chrome, Firefox or Edge.

### Appendix C: User Roles

| Role                      | Permissions                                                                                       |
|---------------------------|---------------------------------------------------------------------------------------------------|
| Student / Faculty / Staff | Report items, search and filter, view the campus map, view match suggestions, submit claims       |
| Admin                     | All of the above, plus verifying claims, marking items as Found or Returned, and managing records |

### Appendix D: Screenshots

The screenshots below are to be taken from the running application and inserted in the boxes provided.

#### 1. Login page

*Insert screenshot here*

## 2. Report a lost or found item

*Insert screenshot here*

## 3. Search and filter results

*Insert screenshot here*

## 4. Interactive campus map (floor view)

*Insert screenshot here*

## 5. Match suggestions

*Insert screenshot here*

## **6. Submit a claim**

*Insert screenshot here*

## **7. Administrator dashboard**

*Insert screenshot here*

## **8. Claim verification and handover**

*Insert screenshot here*